

Semantics of Programming Languages

The 80/20 Revision Guide

Part IB — Cambridge CST

How this guide is organised

The Semantics paper sets two questions per year. Across **eight years of past papers (2018–2025, 16 questions)** the topic frequencies are:

Topic	Appearances	Share
Semantic equivalence (incl. contextual equivalence)	7	44%
Operational semantics & transition systems	6	38%
Subtyping	4	25%
Type systems & typing judgements	5	31%
Store and memory models	3	19%
Evaluation contexts	3	19%
Concurrency, interleaving, mutexes	1	6%
Other (alpha-equivalence, scope, shadowing)	4	25%

80/20 verdict. The course is organised around the languages **L1** $\rightarrow$ **L2** $\rightarrow$ **L3** (basic imperative $\rightarrow$ adds functions $\rightarrow$ adds products/sums/records/references). **Semantic equivalence and operational semantics together carry $\sim 80\%$ of marks.** Subtyping is reliable Q2 material. Evaluation contexts and the store-typing condition appear regularly.

This document is structured as:

1. Tier 1: operational semantics + typing (the language progression $L1 \rightarrow L2 \rightarrow L3$).
2. Tier 2: subtyping, evaluation contexts, semantic/contextual equivalence.
3. Tier 3: concurrency and supporting material.
4. **The complete rule sheet** (Section 4) — every operational and typing rule for L1, L2, L3, and subtyping, colour-coded to show what each language adds. Refer here while revising.
5. Exam checklist.

Colour key (used throughout, but most heavily in Section 4):

■ L1 (base) ■ L2 adds (functions, let, let rec) ■ L3 adds (pairs, sums, records, refs) ■
Subtyping

1 Operational semantics — the language progression

1.1 Transition systems and structural operational semantics (SOS)

Exam-critical

The foundation. A *transition system* is a pair $(Config, \rightarrow)$ with $\rightarrow \subseteq Config \times Config$. SOS defines $\rightarrow$ by giving a set of *inference rules*; the relation is the smallest one closed under them.

Configurations are pairs $\langle e, s \rangle$ of an expression e and a store s (a finite partial function $\mathbb{L} \rightarrow \mathbb{Z}$ in L1/L2; $\mathbb{L} \rightarrow Values$ in L3). A transition $\langle e, s \rangle \rightarrow \langle e', s' \rangle$ is a single computation step.

Values are terminated states. A configuration is *stuck* if e is not a value and $\langle e, s \rangle \not\rightarrow$. The goal of typing is to ensure well-typed programs never get stuck.

Notation. $\rightarrow^*$ is reflexive-transitive closure. $\rightarrow^\omega$ means there is an infinite reduction sequence (the program diverges).

1.2 L1: the imperative core

Syntax.

$$e ::= n \mid b \mid e_1 \text{ op } e_2 \mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \mid \ell := e \mid !\ell \mid \text{skip} \mid e_1; e_2 \mid \text{while } e_1 \text{ do } e_2 \text{ done}$$

Two flavours of rule:

- **Computation rules** actually compute (e.g. OP+, IF1, IF2, Deref, ASSIGN1, SEQ1, WHILE).
- **Context (congruence) rules** push reduction into sub-expressions and dictate evaluation order (e.g. OP1, OP2, SEQ2, IF3, ASSIGN2).

Determinacy of L1. For any $\langle e, s \rangle$, there is at most one $\langle e', s' \rangle$ with $\langle e, s \rangle \rightarrow \langle e', s' \rangle$. Proved by induction on derivations. Determinacy is broken in L3 because REF1 “picks a fresh location” non-deterministically.

1.3 L2: adds functions

Syntax additions. $e ::= \dots \mid \text{fun } x:T \rightarrow e \mid e_1 e_2 \mid x \mid \text{let } x:T = e_1 \text{ in } e_2 \mid \text{let rec } x:T_1 \rightarrow T_2 = (\text{fun } y:T_1 \rightarrow e_1) \text{ in } e_2$

Types additions. $T ::= \dots \mid T_1 \rightarrow T_2$.

Three choices of function semantics:

- **Call-by-value (CBV)** — the default for this course, and for ML/Java/most languages. Evaluate the argument to a value, then substitute. Rule FUN: $\langle (\text{fun } x:T \rightarrow e) v, s \rangle \rightarrow \langle \{v/x\}e, s \rangle$.
- **Call-by-name (CBN)** — substitute the unevaluated argument; evaluate only on use. Basis of Haskell (which uses call-by-need = CBN with sharing). Rule CBN-FUN: $\langle (\text{fun } x:T \rightarrow e) e_2, s \rangle \rightarrow \langle \{e_2/x\}e, s \rangle$.
- **Full beta** — reduce either side, even inside lambdas. The basis of lambda-calculus theory.

Gotcha

CBV vs CBN observably differ in any language with effects. The lecturer’s example: $e = (\text{fun } x:\text{unit} \rightarrow (\ell := 1); x)(\ell := 2)$ starting in $\{\ell \mapsto 0\}$. CBV: the argument $(\ell := 2)$ runs first, then $\ell := 1$, ending with $\ell \mapsto 1$. CBN: the argument is substituted unevaluated; the body runs $\ell := 1$ then evaluates the substituted argument $(\ell := 2)$, ending with $\ell \mapsto 2$.

Variables and binders. $\text{fun } x:T \rightarrow e$ binds x in e ; $\text{let } x:T = e_1 \text{ in } e_2$ binds x in e_2 . We work *up to alpha-equivalence* — bound names can be silently renamed to avoid capture. Substitution $\{e/x\}e'$ is capture-avoiding.

Two key properties of L2.

- **Substitution lemma:** if $\Gamma \vdash e:T$ and $\Gamma, x:T \vdash e':T'$ with $x \notin \text{dom}(\Gamma)$, then $\Gamma \vdash \{e/x\}e':T'$. Used in the type-preservation proof.
- **Normalisation:** in the L2 sub-language without **while** or store, a well-typed closed expression always terminates. (You need refs or recursion through the store to write a divergent term in pure typed L2.)

1.4 L3: adds data and proper references

Syntax additions. Pairs, sums, records, and general references:

$$\begin{aligned}
e ::= & \dots \mid (e_1, e_2) \mid \mathbf{fst} \ e \mid \mathbf{snd} \ e \\
& \mid \mathbf{inl} \ e:T \mid \mathbf{inr} \ e:T \mid \mathbf{match} \ e \ \mathbf{with} \ \mathbf{inl}(x:T_1) \rightarrow e_1 \mid \mathbf{inr}(y:T_2) \rightarrow e_2 \\
& \mid \{lab_1 = e_1, \dots, lab_k = e_k\} \mid e.lab \\
& \mid \mathbf{ref} \ e \mid !e \mid e_1 := e_2 \mid \ell
\end{aligned}$$

Types additions. $T ::= \dots \mid T_1 * T_2 \mid T_1 + T_2 \mid \{lab_1:T_1, \dots, lab_k:T_k\} \mid T \ \mathbf{ref}$.

The reference upgrade. L1/L2 had only $\ell := e$ and $!\ell$ for *pre-existing* locations storing only *integers*. L3 adds:

- **ref** e — allocates a fresh location holding e 's value. *Has to do something at runtime*: each evaluation gives a fresh ℓ , so $(\mathbf{ref} \ 0, \mathbf{ref} \ 0)$ produces two distinct locations.
- Stores now map $\mathbb{L}$ to *any value*, not just $\mathbb{Z}$. Locations can hold functions, pairs, even other locations.
- $!e$ and $e_1 := e_2$ are now general expressions, not restricted to literal ℓ s.

Consequence: “tying the knot” for recursion through the store. Even without **let rec**, you can write recursive functions in L3 by storing a function in a **ref** and having the stored function deref the **ref** to call itself. This is why L3 loses the normalisation theorem L2 enjoyed.

Curry–Howard correspondence. The typing rules for the *pure* fragment of L3 (no refs, no **let rec**) correspond exactly to the rules of natural deduction in intuitionistic propositional logic: $\rightarrow$ as implication, $*$ as conjunction, $+$ as disjunction. “Programs are proofs.”

1.5 Properties: progress, preservation, safety

These three theorems are the spine of every typed language. Drill the statements.

Progress

If $\Gamma \vdash e:T$, e closed, and $\Gamma \vdash s$ (the store is well-typed at Γ), then either e is a value or there exist e', s' with $\langle e, s \rangle \rightarrow \langle e', s' \rangle$.

Read it as: well-typed programs don't get stuck.

Type Preservation (Subject Reduction)

If $\Gamma \vdash e:T$, e closed, $\Gamma \vdash s$, and $\langle e, s \rangle \rightarrow \langle e', s' \rangle$, then e' is closed and for some Γ' with $\text{dom}(\Gamma') \cap \text{dom}(\Gamma) = \emptyset$, $\Gamma, \Gamma' \vdash e':T$ and $\Gamma, \Gamma' \vdash s'$.

Why Γ' ? $\mathbf{REF1}$ allocates a fresh location, growing the type environment. In L1/L2, $\Gamma' = \emptyset$ always.

Read it as: types are preserved by single steps.

Type Safety

If $\Gamma \vdash e:T$, e closed, $\Gamma \vdash s$, and $\langle e, s \rangle \rightarrow^* \langle e', s' \rangle$, then either e' is a value or it can step further.

Proof sketch: Safety = Progress + Preservation. Induct on the length of the reduction sequence using preservation; apply progress at the end.

Well-typed store (the L3 condition $\Gamma \vdash s$):

$$\Gamma \vdash s \iff \text{dom}(\Gamma) = \text{dom}(s) \text{ and for all } \ell \in \text{dom}(s), \text{ if } \Gamma(\ell) = T \ \mathbf{ref} \ \mathbf{then} \ \Gamma \vdash s(\ell):T.$$

In L1/L2 this collapses to $\text{dom}(\Gamma) \subseteq \text{dom}(s)$ since stores hold only integers.

2 Tier 2: subtyping, evaluation contexts, equivalence

2.1 Subtyping

Exam-critical

Tested in 2019-9, 2020-8, 2022-8, 2024-9. The exam questions reliably ask: state the subtype rules, give an example derivation using `SUB`, explain why `T ref` subtyping is unsound (with a witness).

Subsumption is the gateway rule:

$$\text{SUB} \quad \frac{\Gamma \vdash e:T \quad T <: T'}{\Gamma \vdash e:T'}$$

“If e has type T , it can be used wherever a T' is expected, provided T is a subtype of T' .” This breaks unique typing.

Subtype rules.

- **S-REFL** $T <: T$. **S-TRANS** from $T <: T'$ and $T' <: T''$, get $T <: T''$. So $<:$ is a *preorder* (not antisymmetric — record reordering shows two distinct types can be mutually subtyped).
- **S-RECORD-WIDTH**: a record with extra fields on the right is a subtype of the smaller record.

$$\{lab_1:T_1, \dots, lab_k:T_k, lab_{k+1}:T_{k+1}, \dots\} <: \{lab_1:T_1, \dots, lab_k:T_k\}$$

- **S-RECORD-DEPTH**: subtype within fields.

$$\frac{T_1 <: T'_1 \quad \dots \quad T_k <: T'_k}{\{lab_1:T_1, \dots\} <: \{lab_1:T'_1, \dots\}}$$

- **S-RECORD-ORDER**: permuting fields is allowed (some languages reject this).
- **S-FUN**: *contravariant in argument, covariant in result*.

$$\frac{T'_1 <: T_1 \quad T_2 <: T'_2}{T_1 \rightarrow T_2 <: T'_1 \rightarrow T'_2}$$

- **S-PAIR**: covariant in both components.

Gotcha

`T ref` subtyping is unsound either way.

- *Covariant* ($T <: T' \Rightarrow T \text{ ref} <: T' \text{ ref}$) breaks: cast a `{p:int,q:int} ref` to `{p:int} ref` (legal by covariance + **S-RECORD-WIDTH**), assign `{p=0}` through it, then `deref` via the original two-field type and read a missing `q`.
- *Contravariant* breaks symmetrically.

The correct rule: **ref** is *invariant* — $T \text{ ref} <: T' \text{ ref}$ only when $T = T'$.

Why functions are contravariant. If $f : T_1 \rightarrow T_2$ is used where a $T'_1 \rightarrow T'_2$ is expected, callers will pass it T'_1 values, so f must accept those: $T'_1 <: T_1$. Callers will use the result as T'_2 , so f ’s output (a T_2) must be usable as T'_2 : $T_2 <: T'_2$.

Simple object encoding. An object is a record whose fields are functions sharing a closed-over reference for state. Method-class subtyping falls out of record-width subtyping: “ResetCounter $<:$ Counter” because the former has all the latter’s methods plus more.

2.2 Evaluation contexts — a presentation refactor

The L3 semantics has many congruence rules (OP1, OP2, SEQ2, IF3, APP1, APP2, LET1, PAIR1, PAIR2, ...). **Evaluation contexts** bundle them all into one rule.

Definition. An evaluation context E is an expression with a single $\square$ “hole” marking where the next reduction can happen, given by:

$$\begin{aligned} E ::= & \square \mid \square \text{ op } e \mid v \text{ op } \square \mid \text{if } \square \text{ then } e \text{ else } e \mid \square; e \mid \square e \mid v \square \\ & \mid \text{let } x:T = \square \text{ in } e \mid (\square, e) \mid (v, \square) \mid \text{fst } \square \mid \text{snd } \square \\ & \mid \text{inl } \square:T \mid \text{inr } \square:T \mid \text{match } \square \text{ with } \dots \\ & \mid \{lab_1 = v_1, \dots, lab_i = \square, \dots, lab_k = e_k\} \mid \square.lab \\ & \mid \square := e \mid v := \square \mid !\square \mid \text{ref } \square \end{aligned}$$

The single context rule:

$$\text{EVAL} \quad \frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle E[e], s \rangle \rightarrow \langle E[e'], s' \rangle}$$

Then keep all the computation rules, drop all the congruence rules. The two presentations define exactly the same $\rightarrow$ relation (Theorem 26 in the notes).

Why use it. The grammar of E fully specifies evaluation order in one place. Adding a new construct just requires adding one E form per “hole” position and the computation rule. Essential for large languages.

2.3 Semantic and contextual equivalence

Exam-critical

The single most-tested topic (7/16 questions). Memorise the L1 definition, the L3 contextual-equivalence definition, the four “good equivalence” criteria, and the canonical examples.

L1: a simple definition that works

For typed L1, define $e_1 \simeq_{\Gamma}^T e_2$ to hold iff $\Gamma \vdash e_1 : T$, $\Gamma \vdash e_2 : T$, and for all s with $\text{dom}(\Gamma) \subseteq \text{dom}(s)$, either:

- (a) both diverge: $\langle e_1, s \rangle \rightarrow^{\omega}$ and $\langle e_2, s \rangle \rightarrow^{\omega}$; or
- (b) both terminate at the same value *and* the same store: $\langle e_1, s \rangle \rightarrow^* \langle v, s' \rangle$ and $\langle e_2, s \rangle \rightarrow^* \langle v, s' \rangle$.

Why “same store too”? If you allowed e_1, e_2 to leave different stores, the context $\square; !\ell$ would distinguish them, breaking congruence.

Theorem (Congruence for L1). $\simeq_{\Gamma}^T$ is a congruence: $e_1 \simeq_{\Gamma}^T e_2 \Rightarrow C[e_1] \simeq_{\Gamma}^{T'} C[e_2]$ for every context C that types both sides. Proved by case analysis on C .

L3: simple definition fails, use contextual equivalence

For L3, the obvious value-and-store definition is *not* a congruence. So we define equivalence *by* the congruence property:

Contextual equivalence (L3). Typed L3 programs $\Gamma \vdash e_1 : T$, $\Gamma \vdash e_2 : T$ are contextually equivalent iff for every context C with $\{\} \vdash C[e_1] : \text{unit}$ and $\{\} \vdash C[e_2] : \text{unit}$, either:

- (a) $\langle C[e_1], \{\} \rangle \rightarrow^{\omega}$ and $\langle C[e_2], \{\} \rangle \rightarrow^{\omega}$; or
- (b) both reduce to $\langle \text{skip}, s_1 \rangle$ and $\langle \text{skip}, s_2 \rangle$ (note: *any* terminating stores — we observe only termination).

Congruence is automatic by construction. Contextual equivalence is the largest congruence preserving termination. The trade-off: it's *undecidable in general*, and very hard to prove particular equivalences. Research-active topic.

The four criteria a “good” equivalence must satisfy

1. Programs with observably-different values are not equivalent.
2. Terminating $\not\approx$ diverging.
3. $\simeq$ is an equivalence relation (reflexive, symmetric, transitive).
4. $\simeq$ is a congruence: $e_1 \simeq e_2 \Rightarrow C[e_1] \simeq C[e_2]$ for all C .

And subject to all those: $\simeq$ should be as coarse as possible (relate as many programs as possible).

Canonical examples to know cold

Pair	Equivalent?	Why
$2 + 2$ vs 4	yes	both reduce to 4 in any store.
$\ell := 0; 4$ vs $\ell := 1; 3 + !\ell$	no	distinguished by $C[\Box] = \Box + !\ell$
$(\ell := !\ell + 1); (\ell := !\ell - 1)$ vs $(\ell := !\ell)$	yes	both leave the store unchanged.
$e_1; e_2$ vs $e_2; e_1$	not in general	only if neither touches the store.

Watch for the trick: two programs that always terminate at the same value can be *inequivalent* if they leave different stores — a context that reads the store separates them.

3 Tier 3: concurrency

The notes' concluding lecture adds parallel composition to L1, giving a tiny concurrent language.

Syntax addition. $e ::= \dots \mid e_1 \parallel e_2$. **Types addition.** $T ::= \dots \mid \text{proc}$.

Typing. $\text{THREAD: } \frac{\Gamma \vdash e : \text{unit}}{\Gamma \vdash e : \text{proc}}$ and $\text{PARALLEL: } \frac{\Gamma \vdash e_1 : \text{proc} \quad \Gamma \vdash e_2 : \text{proc}}{\Gamma \vdash e_1 \parallel e_2 : \text{proc}}$.

Reduction. Symmetric: either thread can step.

$$\text{PARALLEL1} \quad \frac{\langle e_1, s \rangle \rightarrow \langle e'_1, s' \rangle}{\langle e_1 \parallel e_2, s \rangle \rightarrow \langle e'_1 \parallel e_2, s' \rangle} \quad \text{PARALLEL2} \quad \frac{\langle e_2, s \rangle \rightarrow \langle e'_2, s' \rangle}{\langle e_1 \parallel e_2, s \rangle \rightarrow \langle e_1 \parallel e'_2, s' \rangle}$$

Loss of determinacy. Even $\langle (\ell := 1) \parallel (\ell := 2), \{\ell \mapsto 0\} \rangle$ can reach $\{\ell \mapsto 1\}$ or $\{\ell \mapsto 2\}$.

Atomicity. Assignment and dereference are atomic in this model (you don't get half-words torn). *Compound* expressions like $\ell := !\ell + 1$ are *not* atomic — a race in $\ell := !\ell + 1 \parallel \ell := !\ell + 7$ can lose updates.

Mutexes. Add **lock** m and **unlock** m and an extra mutex-state $M : \mathbb{M} \rightarrow \mathbb{B}$ in configurations $\langle e, s, M \rangle$. Rule **LOCK** only fires when $\neg M(m)$ — atomically test, set, and proceed.

Two-phase locking (2PL) sufficient condition for serialisability: each transaction has an expanding phase (acquires locks, no releases) then a shrinking phase (releases, no acquires). Strict 2PL holds all locks until commit. Acquire locks in a fixed global order to avoid deadlock.

4 Complete rule sheet: L1, L2, L3, subtyping

Colour key. ■ L1 base rules ■ added by L2 (functions, let, let rec) ■ added by L3 (pairs, sums, records, refs) ■ subtyping

Conventions: $n \in \mathbb{Z}$, $b \in \{\text{true}, \text{false}\}$, v ranges over values, s over stores, Γ over type environments, $op \in \{+, \geq\}$.

Values

$v ::= b \mid n \mid \text{skip} \quad | \text{fun } x:T \rightarrow e \quad | (v_1, v_2) \mid \text{inl } v:T \mid \text{inr } v:T \mid \{lab_1 = v_1, \dots, lab_k = v_k\} \mid \ell$

Operational semantics (reduction)

Arithmetic and Booleans (L1)

OP+ $\langle n_1 + n_2, s \rangle \rightarrow \langle n, s \rangle$ if $n = n_1 + n_2$

OP $\geq$ $\langle n_1 \geq n_2, s \rangle \rightarrow \langle b, s \rangle$ if $b = (n_1 \geq n_2)$

OP1 $\frac{\langle e_1, s \rangle \rightarrow \langle e'_1, s' \rangle}{\langle e_1 \text{ op } e_2, s \rangle \rightarrow \langle e'_1 \text{ op } e_2, s' \rangle}$

OP2 $\frac{\langle e_2, s \rangle \rightarrow \langle e'_2, s' \rangle}{\langle v \text{ op } e_2, s \rangle \rightarrow \langle v \text{ op } e'_2, s' \rangle}$

Store (L1) — L1 form only handles integer locations.

DEREF $\langle !\ell, s \rangle \rightarrow \langle n, s \rangle$ if $s(\ell) = n$

ASSIGN1 $\langle \ell := n, s \rangle \rightarrow \langle \text{skip}, s + \{\ell \mapsto n\} \rangle$

ASSIGN2 $\frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle \ell := e, s \rangle \rightarrow \langle \ell := e', s' \rangle}$

Sequencing (L1)

SEQ1 $\langle \text{skip}; e_2, s \rangle \rightarrow \langle e_2, s \rangle$

SEQ2 $\frac{\langle e_1, s \rangle \rightarrow \langle e'_1, s' \rangle}{\langle e_1; e_2, s \rangle \rightarrow \langle e'_1; e_2, s' \rangle}$

Conditional (L1)

IF1 $\langle \text{if true then } e_2 \text{ else } e_3, s \rangle \rightarrow \langle e_2, s \rangle$

IF2 $\langle \text{if false then } e_2 \text{ else } e_3, s \rangle \rightarrow \langle e_3, s \rangle$

IF3 $\frac{\langle e_1, s \rangle \rightarrow \langle e'_1, s' \rangle}{\langle \text{if } e_1 \text{ then } e_2 \text{ else } e_3, s \rangle \rightarrow \langle \text{if } e'_1 \text{ then } e_2 \text{ else } e_3, s' \rangle}$

While loop (L1)

WHILE $\langle \text{while } e_1 \text{ do } e_2 \text{ done}, s \rangle \rightarrow$

$\langle \text{if } e_1 \text{ then } (e_2; \text{while } e_1 \text{ do } e_2 \text{ done}) \text{ else skip}, s \rangle$

Functions (CBV) (L2)

APP1 $\frac{\langle e_1, s \rangle \rightarrow \langle e'_1, s' \rangle}{\langle e_1 e_2, s \rangle \rightarrow \langle e'_1 e_2, s' \rangle}$

APP2 $\frac{\langle e_2, s \rangle \rightarrow \langle e'_2, s' \rangle}{\langle v e_2, s \rangle \rightarrow \langle v e'_2, s' \rangle}$

FUN $\langle (\text{fun } x:T \rightarrow e) v, s \rangle \rightarrow \langle \{v/x\}e, s \rangle$

Local definitions (L2)

LET1 $\frac{\langle e_1, s \rangle \rightarrow \langle e'_1, s' \rangle}{\langle \text{let } x:T = e_1 \text{ in } e_2, s \rangle \rightarrow \langle \text{let } x:T = e'_1 \text{ in } e_2, s' \rangle}$

LET2 $\langle \text{let } x:T = v \text{ in } e_2, s \rangle \rightarrow \langle \{v/x\}e_2, s \rangle$

LETRECFUN $\langle \text{let rec } x:T_1 \rightarrow T_2 = (\text{fun } y:T_1 \rightarrow e_1) \text{ in } e_2, s \rangle \rightarrow$

$\langle \{(\text{fun } y:T_1 \rightarrow \text{let rec } x \dots \text{ in } e_1)/x\}e_2, s \rangle$

Pairs (L3)

PAIR1 $\frac{\langle e_1, s \rangle \rightarrow \langle e'_1, s' \rangle}{\langle (e_1, e_2), s \rangle \rightarrow \langle (e'_1, e_2), s' \rangle}$

$$\begin{array}{c}
\text{PAIR2} \frac{\langle e_2, s \rangle \rightarrow \langle e'_2, s' \rangle}{\langle (v_1, e_2), s \rangle \rightarrow \langle (v_1, e'_2), s' \rangle} \\
\text{PROJ1} \frac{}{\langle \mathbf{fst}(v_1, v_2), s \rangle \rightarrow \langle v_1, s \rangle} \\
\text{PROJ2} \frac{}{\langle \mathbf{snd}(v_1, v_2), s \rangle \rightarrow \langle v_2, s \rangle} \\
\text{PROJ3} \frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle \mathbf{fst} \ e, s \rangle \rightarrow \langle \mathbf{fst} \ e', s' \rangle} \\
\text{PROJ4} \frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle \mathbf{snd} \ e, s \rangle \rightarrow \langle \mathbf{snd} \ e', s' \rangle}
\end{array}$$

Sums (L3)

$$\begin{array}{c}
\text{INL} \frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle \mathbf{inl} \ e:T, s \rangle \rightarrow \langle \mathbf{inl} \ e':T, s' \rangle} \\
\text{INR} \frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle \mathbf{inr} \ e:T, s \rangle \rightarrow \langle \mathbf{inr} \ e':T, s' \rangle}
\end{array}$$

$$\text{MATCH1} \frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle \mathbf{match} \ e \ \mathbf{with} \ \dots, s \rangle \rightarrow \langle \mathbf{match} \ e' \ \mathbf{with} \ \dots, s' \rangle}$$

$$\text{MATCH2} \langle \mathbf{match} \ \mathbf{inl} \ v:T \ \mathbf{with} \ \mathbf{inl}(x:T_1) \rightarrow e_1 \mid \mathbf{inr}(y:T_2) \rightarrow e_2, s \rangle \rightarrow \langle \{v/x\}e_1, s \rangle$$

$$\text{MATCH3} \langle \mathbf{match} \ \mathbf{inr} \ v:T \ \mathbf{with} \ \dots, s \rangle \rightarrow \langle \{v/y\}e_2, s \rangle$$

∞

Records (L3)

$$\text{RECORD1} \frac{\langle e_i, s \rangle \rightarrow \langle e'_i, s' \rangle}{\langle \{\dots, \mathit{lab}_i = e_i, \dots\}, s \rangle \rightarrow \langle \{\dots, \mathit{lab}_i = e'_i, \dots\}, s' \rangle}$$

Typing rules ($\Gamma \vdash e : T$)

Constants and operators (L1)

$$\text{INT} \ \Gamma \vdash n : \mathbf{int} \quad (n \in \mathbb{Z})$$

$$\text{BOOL} \ \Gamma \vdash b : \mathbf{bool} \quad (b \in \{\mathbf{true}, \mathbf{false}\})$$

$$\text{SKIP} \ \Gamma \vdash \mathbf{skip} : \mathbf{unit}$$

$$\text{OP}+ \frac{\Gamma \vdash e_1 : \mathbf{int} \quad \Gamma \vdash e_2 : \mathbf{int}}{\Gamma \vdash e_1 + e_2 : \mathbf{int}}$$

$$\text{OP}\geq \frac{\Gamma \vdash e_1 : \mathbf{int} \quad \Gamma \vdash e_2 : \mathbf{int}}{\Gamma \vdash e_1 \geq e_2 : \mathbf{bool}}$$

(leftmost non-value field)

$$\text{RECORD2} \ \langle \{\mathit{lab}_1 = v_1, \dots, \mathit{lab}_k = v_k\}. \mathit{lab}_i, s \rangle \rightarrow \langle v_i, s \rangle$$

$$\text{RECORD3} \frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle e. \mathit{lab}_i, s \rangle \rightarrow \langle e'. \mathit{lab}_i, s' \rangle}$$

General references (L3 — replaces L1 store rules)

$$\text{REF1} \ \langle \mathbf{ref} \ v, s \rangle \rightarrow \langle \ell, s + \{\ell \mapsto v\} \rangle, \quad \ell \notin \text{dom}(s)$$

$$\text{REF2} \frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle \mathbf{ref} \ e, s \rangle \rightarrow \langle \mathbf{ref} \ e', s' \rangle}$$

$$\text{DEREF1} \ \langle !\ell, s \rangle \rightarrow \langle v, s \rangle, \quad s(\ell) = v$$

$$\text{DEREF2} \frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle !e, s \rangle \rightarrow \langle !e', s' \rangle}$$

$$\text{ASSIGN1} \ \langle \ell := v, s \rangle \rightarrow \langle \mathbf{skip}, s + \{\ell \mapsto v\} \rangle$$

$$\text{ASSIGN2} \frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle \ell := e, s \rangle \rightarrow \langle \ell := e', s' \rangle}$$

$$\text{ASSIGN3} \frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle e := e_2, s \rangle \rightarrow \langle e' := e_2, s' \rangle}$$

Control flow (L1)

$$\text{IF} \frac{\Gamma \vdash e_1 : \mathbf{bool} \quad \Gamma \vdash e_2 : T \quad \Gamma \vdash e_3 : T}{\Gamma \vdash \mathbf{if} \ e_1 \ \mathbf{then} \ e_2 \ \mathbf{else} \ e_3 : T}$$

$$\text{SEQ} \frac{\Gamma \vdash e_1 : \mathbf{unit} \quad \Gamma \vdash e_2 : T}{\Gamma \vdash e_1 ; e_2 : T}$$

$$\text{WHILE} \frac{\Gamma \vdash e_1 : \mathbf{bool} \quad \Gamma \vdash e_2 : \mathbf{unit}}{\Gamma \vdash \mathbf{while} \ e_1 \ \mathbf{do} \ e_2 \ \mathbf{done} : \mathbf{unit}}$$

Store (L1 form)

$$\text{ASSIGN} \frac{\Gamma(\ell) = \mathbf{intref} \quad \Gamma \vdash e : \mathbf{int}}{\Gamma \vdash \ell := e : \mathbf{unit}}$$

$$\text{DEREF} \frac{\Gamma(\ell) = \text{intref}}{\Gamma \vdash !\ell : \text{int}}$$

Functions (L2)

$$\begin{array}{c} \text{VAR} \Gamma \vdash x : T \quad \text{if } \Gamma(x) = T \\ \\ \text{FUN} \frac{\Gamma, x:T \vdash e : T'}{\Gamma \vdash \text{fun } x:T \rightarrow e : T \rightarrow T'} \\ \\ \text{APP} \frac{\Gamma \vdash e_1 : T \rightarrow T' \quad \Gamma \vdash e_2 : T}{\Gamma \vdash e_1 e_2 : T'} \end{array}$$

Local definitions (L2)

$$\begin{array}{c} \text{LET} \frac{\Gamma \vdash e_1 : T \quad \Gamma, x:T \vdash e_2 : T'}{\Gamma \vdash \text{let } x:T = e_1 \text{ in } e_2 : T'} \\ \\ \text{LETRECFUN} \frac{\Gamma, x:T_1 \rightarrow T_2, y:T_1 \vdash e_1 : T_2 \quad \Gamma, x:T_1 \rightarrow T_2 \vdash e_2 : T}{\Gamma \vdash \text{let rec } x:T_1 \rightarrow T_2 = (\text{fun } y:T_1 \rightarrow e_1) \text{ in } e_2 : T} \end{array}$$

Pairs (L3)

$$\begin{array}{c} \text{PAIR} \frac{\Gamma \vdash e_1 : T_1 \quad \Gamma \vdash e_2 : T_2}{\Gamma \vdash (e_1, e_2) : T_1 * T_2} \\ \\ \text{PROJ1} \frac{\Gamma \vdash e : T_1 * T_2}{\Gamma \vdash \text{fst } e : T_1} \\ \\ \text{PROJ2} \frac{\Gamma \vdash e : T_1 * T_2}{\Gamma \vdash \text{snd } e : T_2} \end{array}$$

Sums (L3)

$$\begin{array}{c} \text{INL} \frac{\Gamma \vdash e : T_1}{\Gamma \vdash \text{inl } e : T_1 + T_2 : T_1 + T_2} \\ \\ \text{INR} \frac{\Gamma \vdash e : T_2}{\Gamma \vdash \text{inr } e : T_1 + T_2 : T_1 + T_2} \\ \\ \text{MATCH} \frac{\Gamma \vdash e : T_1 + T_2 \quad \Gamma, x:T_1 \vdash e_1 : T \quad \Gamma, y:T_2 \vdash e_2 : T}{\Gamma \vdash \text{match } e \text{ with inl}(x:T_1) \rightarrow e_1 \mid \text{inr}(y:T_2) \rightarrow e_2 : T} \end{array}$$

Records (L3)

$$\text{RECORD} \frac{\Gamma \vdash e_1 : T_1 \quad \dots \quad \Gamma \vdash e_k : T_k}{\Gamma \vdash \{lab_1 = e_1, \dots, lab_k = e_k\} : \{lab_1:T_1, \dots, lab_k:T_k\}}$$

Stores and configurations

$$\text{RECORDPROJ} \frac{\Gamma \vdash e : \{lab_1:T_1, \dots, lab_k:T_k\}}{\Gamma \vdash e.lab_i : T_i}$$

General references (L3)

$$\begin{array}{c} \text{REF} \frac{\Gamma \vdash e : T}{\Gamma \vdash \text{ref } e : T \text{ref}} \\ \\ \text{ASSIGN} \frac{\Gamma \vdash e_1 : T \text{ref} \quad \Gamma \vdash e_2 : T}{\Gamma \vdash e_1 := e_2 : \text{unit}} \\ \\ \text{DEREF} \frac{\Gamma \vdash e : T \text{ref}}{\Gamma \vdash !e : T} \\ \\ \text{LOC} \frac{\Gamma(\ell) = T \text{ref}}{\Gamma \vdash \ell : T \text{ref}} \end{array}$$

Subtyping (extends L3)

$$\begin{array}{c} \text{SUB} \frac{\Gamma \vdash e : T \quad T <: T'}{\Gamma \vdash e : T'} \\ \\ \text{S-REFL} T <: T \\ \\ \text{S-TRANS} \frac{T <: T' \quad T' <: T''}{T <: T''} \\ \\ \text{S-RECORD-WIDTH} \end{array}$$

$$\{lab_1:T_1, \dots, lab_k:T_k, lab_{k+1}:T_{k+1}, \dots\} <: \{lab_1:T_1, \dots, lab_k:T_k\}$$

$$\text{S-RECORD-DEPTH} \frac{T_1 <: T'_1 \quad \dots \quad T_k <: T'_k}{\{lab_1:T_1, \dots, lab_k:T_k\} <: \{lab_1:T'_1, \dots, lab_k:T'_k\}}$$

$$\text{S-RECORD-ORDER} \{lab_1:T_1, \dots, lab_k:T_k\} <: \{lab_{\pi(1)}:T_{\pi(1)}, \dots\}$$

$$(\pi \text{ a permutation of } 1, \dots, k)$$

$$\text{S-FUN} \frac{T'_1 <: T_1 \quad T_2 <: T'_2}{T_1 \rightarrow T_2 <: T'_1 \rightarrow T'_2}$$

$$\text{S-PAIR} \frac{T_1 <: T'_1 \quad T_2 <: T'_2}{T_1 * T_2 <: T'_1 * T'_2}$$

(refs are *invariant* — no sound subtype rule)

- **L1/L2 stores:** finite partial functions $s : \mathbb{L} \rightarrow \mathbb{Z}$ (locations hold integers).
- **L3 stores:** finite partial functions $s : \mathbb{L} \rightarrow \text{Values}$ (locations hold *any* value, including functions and other locations).
- **Type-environment role:** in L1/L2, Γ maps locations to **intref** only. The condition $\text{dom}(\Gamma) \subseteq \text{dom}(s)$ suffices for safety.
- **L2 adds:** Γ also maps variables to types. Type-environments are pairs $(\Gamma_{loc}, \Gamma_{var})$.
- **L3 adds:** Γ maps locations to $T\text{ref}$ for any T . The store must be well-typed at Γ ($\Gamma \vdash s$): for each ℓ , the stored value must have type matching $\Gamma(\ell)$.
Type preservation may extend Γ with fresh locations.

5 The one-page exam checklist

Exam-critical

If you have one hour to revise, do these in order:

1. Be able to give a small reduction sequence using L1's `OP1/OP2/SEQ/WHILE` rules.
2. Be able to give a typing derivation $\Gamma \vdash e : T$ for any small L1 program.
3. State Progress, Type Preservation, and Safety, and explain how Safety = Progress + Preservation.
4. Explain CBV vs CBN with the standard $\ell := 2$ example showing they differ in any language with effects.
5. Be able to apply `FUN`, `APP1`, `APP2` to reduce a small lambda example to a value.
6. Be able to give the L3 reduction rules for `ref`, `!e`, and $e_1 := e_2$, and explain why determinacy is lost.
7. State the well-typed store condition $\Gamma \vdash s$ and explain why L3 type preservation extends Γ with a fresh Γ' .
8. Give the subtype rules, paying attention to `S-FUN` contravariance and the invariance of T_{ref} . Be able to construct the standard T_{ref} -subtyping counterexample.
9. Reformulate the L3 semantics using evaluation contexts $E + \text{EVAL}$.
10. State the L1 definition of semantic equivalence $\simeq_{\Gamma}^T$, give the four “good equivalence” criteria, and explain why the L3 simple definition fails to be a congruence.
11. State contextual equivalence for L3 and explain why it's a congruence by construction.
12. Know the canonical equivalent and non-equivalent pairs ($2 + 2 \simeq 4$; $\ell := 0; 4 \not\simeq \ell := 1; 3 + !\ell$; $\ell := !\ell + 1$; $\ell := !\ell - 1 \simeq \ell := !\ell$).
13. For concurrency: explain how the parallel rule loses determinacy, why $\ell := !\ell + 1$ races even with atomic assignment, and how a mutex fixes it.

Good luck.